Traffic Master

A Mini Project Report Submitted

In

EE - VLSI

by

Shivam Singh (2401331730053)
Saurav Shashank (2401331730049)
Satyam Yadav (2401331730048)
Aditya Raj (2401331730005)
Rishu Raj Singh (2401331730043)

Under the Supervision of
Mr. Manoj Prajapati
Assistance Professor, Training Department

VLSI Department
School Name
NOIDA INSTITUTE OF ENGINEERING AND TECHNOLOGY, GREATER NOIDA
(An Autonomous Institute)
Affiliated to
DR. A.P.J. ABDUL KALAM TECHNICAL UNIVERSITY, LUCKNOW

November, 2025

DECLARATION

We hereby declare that the work presented in this report entitled "MINI **Traffic Master**", was carried out by us. We have not submitted the matter embodied in this report for the award of any other degree or diploma of any other University or Institute. We have given due credit to the original authors/sources for all the words, ideas, diagrams, graphics, computer programs, experiments, results, that are not my original contribution. We have used quotation marks to identify verbatim sentences and given credit to the original authors/sources.

We affirm that no portion of our work is plagiarized, and the experiments and results reported in the report are not manipulated. In the event of a complaint of plagiarism and the manipulation of the experiments and results, we shall be fully responsible and answerable.

Name : Shivam Singh
Roll Number : 2401331730053

Name : Saurav Shashank
Roll Number : 2401331730049

Name : Satyam Yadav
Roll Number : 2401331730048

Name : Aditya Raj
Roll Number : 2401331730005

Name : Rishu Raj Singh
Roll Number : 2401331730043

CERTIFICATE

Certified that Shivam Singh (Roll No : 2401331730053), Saurav Shashank (Roll No : 2401331730049), Satyam Yadav (Roll No : 2401331730048), Aditya Raj (Roll No : 240133173000005) and Rishu Raj Singh (Roll No : 2401331730043) have carried out the research work presented in this Project Report entitled "Traffic Master Report" in partial fulfilment of the requirements for the award of the Bachelor of Technology in Computer Science & Engineering from Dr. A.P.J. Abdul Kalam Technical University, Lucknow, under our supervision. The Project Report embodies results of original work, and studies are carried out by the students herself/himself. The contents of the Project Report do not form the basis for the award of any other degree to the candidate or to anybody else from this or any other University/Institution.

Signature

Manoj Prajapati
(Assistant Professor)

NIET Greater Noida

Signature

Rajan Singh
(HOD – VLSI Department)

NIET Greater Noida

ACKNOWLEDGEMENTS

We would like to express my gratitude towards Mr. Manoj Prajapati for his guidance and constant supervision as well as for providing necessary information regarding the project & also for their support in completing the project.

Our thanks and appreciations to respected Dr. Rajan Singh, for their motivation and support throughout.

ABSTRACT

Traffic Master is a console-based reflex and decision-making game developed in C++. The player acts as a traffic signal controller, managing vehicles, pedestrians, and emergency situations at a busy intersection. Using random event generation, timing functions, and conditional logic, the game simulates real-world traffic challenges under time pressure.

The objective is to ensure smooth traffic flow while preventing accidents and congestion. Players earn points for correct decisions and lose points for wrong or delayed responses. The project demonstrates practical use of loops, conditions, randomization, timing, and file handling in C++.

Overall, Traffic Master is a simple yet engaging project that combines logic, timing, and interactivity, showcasing the fundamentals of real-time programming and problem-solving.

TABLE OF CONTENTS

	Page No.
Declaration	i
Certificate	ii
Acknowledgements	iii
Abstract	iv

CHAPTER 1: INTRODUCTION

1.1 BACKGROUND

2. IDENTIFIED ISSUES/RESEARCH GAPS
3. OBJECTIVE AND SCOPE
4. PROJECT REPORT ORGANIZATION

CHAPTER 2: REQUIEMENT AND ANALYSIS

1. Requirement Specification
2. System Study / Feasibility Study
3. Planning and Scheduling
4. Hardware and Software Requirements
5. Preliminary Product Description

CHAPTER 3: SYSTEM DESIGN

1. Software Development Life Cycle (SDLC) Model Used
2. System Architecture
3. UML Diagrams
   - Use Case Diagram
   - Class Diagram
   - Activity Diagram
   - Sequence Diagram
4. Flowchart of the System

CHAPTER 4: IMPLEMENTATION AND RESULTS

1. Implementation Details
2. Screenshots / Output Explanation
3. Performance Evaluation

CHAPTER 5: CONCLUSION AND FUTURE WORK

1. Summary of Work
2. Limitations
3. Future Enhancements

REFERENCES

APPENDICES

CURRICULUM VITA

PROJECT REPORT ORGANIZATION

- Chapter 1 covers game context, motivation, objectives, and synopsis structure.
- Chapter 2 describes requirements and feasibility.
- Chapter 3 details system design and architecture.
- Chapter 4 presents implementation results.
- Chapter 5 lists conclusions and suggested extensions.

CHAPTER 1

CHAPTER 1: INTRODUCTION

1. BACKGROUND

1. History and Significance of Traffic master

The idea for Traffic Master originated from the real-world challenges of urban traffic control and the increasing need for quick decision-making in busy intersections. The concept was inspired by observing how human signal controllers and automated systems handle unpredictable traffic conditions.

The project was designed to simulate this environment in a simple, interactive, and educational way using C++. The goal was to create a text-based game that demonstrates how logical programming and timing functions can be used to represent real-life decision systems. Over time, the project evolved into a complete console-based reflex game featuring scoring, random events, and a high-score system, making it both instructive and entertaining.

2. Relevance of Console-Based Game Projects

Developing console games enhances understanding of programming concepts like input/output, control structures, and data usage. C is often chosen for such foundational assignments thanks to its low-level capabilities and educational value in understanding procedural logic.

2. Learning Outcomes through Project Implementation

Through the Traffic Master project, the following outcomes were achieved:

- Practical understanding of C++ concepts like loops, conditions, and file handling.
- Application of randomization and timing functions in real-time situations.
- Improved logical thinking and decision-making under time constraints.
- Experience in developing an interactive console-based application.
- Better understanding of real-world simulation using programming logic.

1. IDENTIFIED ISSUES/RESEARCH GAPS

Common issues encountered in such projects include:

- Most traffic master simulations are complex and require graphical interfaces or high end systems.
- Beginner-level C++ projects rarely include real-time decision-making or timing logic.
- There is limited focus on applying core programming systems like traffic control.

- Few educational projects help students improve both programming skills and reflex-based logical thinking.

1.3 OBJECTIVE AND SCOPE

The key objectives:

- To design and develop a real-time, reflex-based console game using C++.
- To simulate traffic control scenarios involving cars, pedestrians, and emergency vehicles.
- To apply core programming concepts such as loops, conditional statements, randomization, and file handling.
- To enhance quick decision-making and logical thinking through interactive gameplay.

Scope:

The project focuses on a text-based traffic control simulation that runs on PC or mobile terminals. It aims to demonstrate how simple logic and timing functions can replicate real-life decision systems. The game is lightweight, easy to use, and serves as a learning tool for beginners in programming.

CHAPTER 2: REQUIREMENT AND ANALYSIS

1. Requirement Specification

Functional requirements:

- Displaying a 3x3 game board
- Taking user moves from the keyboard
- Validating each move

Non-functional requirements:

- Should be fast, reliable, and user-friendly
- Compatibility with standard C compilers on Windows or Linux systems.

2. System Study/Feasibility Study

The project is feasible for academic timelines. Only basic hardware (PC) and open-source C compiler are needed. The game logic mirrors the playable board and is conceptually simple, making implementation approachable for students.

3. Planning and Scheduling

Suggested timeline:

- Week 1: Requirement and planning
- Week 2: Coding and function breakdown
- Week 3: Testing/Debugging
- Week 4: Documentation and final presentation

4. Hardware and Software Requirements

- Hardware: Standard PC
- Software: GCC/MinGW or Code::Blocks IDE for C programming
- OS: Windows/Linux

5. Preliminary Product Description

The output will be a console program that displays the board, prompts for user moves, validates input, and outputs the winner or a draw at the game's end. The game will feature clear prompts and instructions for ease of use.

CHAPTER 3: SYSTEM DESIGN

3.1 Software Development Life Cycle (SDLC) Model Used

The Traffic Master project follows the Waterfall Model of software development.

This model was chosen because it provides a clear, linear structure for project execution, which is ideal for small-scale academic projects.

Phases of the Waterfall Model:

1. Requirement Analysis: Understanding the problem, defining the objectives, and identifying the game logic.

2. System Design: Designing the program structure, control flow, and event-handling logic.

3. Implementation: Writing C++ code using loops, conditions, randomization, and file handling.

4. Testing: Checking all possible gameplay situations for correctness and stability.

5. Deployment & Maintenance: Finalizing the game for submission and future improvements.

3.2 System Architecture

The architecture of Traffic Master is designed around a modular and event-driven approach, where each module performs a specific task in the game.

Main Components:

- Input Module: Handles player responses (G/R) within time limits.
- Event Generator: Produces random conditions such as number of cars, pedestrians, and emergencies.
- Decision Logic Module: Evaluates user input and determines the outcome (safe move, accident, or delay).
- Score and Status Module: Updates and displays score, accidents, and backlog after each round.
- File Handling Module: Saves and loads high scores from a text file.

This modular design ensures clarity, easy debugging, and scalability for future upgrades.

3.3 UML Diagrams

Actors:

- Player
- System

Description:

The player interacts with the system by choosing when to turn the signal GREEN or RED. The system evaluates this decision, updates the score, and generates new traffic conditions.

Main Use Cases:

- Start Game
- Display Random Events
- Accept Player Input
- Evaluate Decision
- Update Score and Display Results
- Save/Load High Score

Activity Diagram

- Start Game
- Generate Random Events
- Display Traffic Situation
- Wait for Player Input (within 3 seconds)
- Evaluate Input → Safe / Accident / Delay
- Update Scores and Backlog
- Check Game Over Conditions
- Display Results → Continue or End

3.4 Flowchart of the System

Flow of the Game:

```
START
        |
    Initialize Variables
  (score, accidents, backlog)
        |
    Generate Random Events
 (cars, pedestrians, emergency)
        |
    Take User Input (G/R)
        |
  Evaluate Decision:
  ┌──────────────────────────┐
  │ Correct → +Points        │
  │ Wrong → +Accident        │
  │ Delay → +Backlog         │
  └──────────────────────────┘
        |
  Display Round Summary
        |
 Check: Accidents < 3 and Backlog < 20 ?
        |
      Yes → Next Round
        |
      No → Game Over
        |
  Display Final Score
```

|

Save High Score

|

END

3.5 System Design Summary

The design of Traffic Master emphasizes simplicity, modularity, and real-time interactivity. Each component performs a defined function, contributing to the smooth execution of the overall program.

The combination of randomization, timing control, and decision logic provides a realistic simulation of traffic management in a minimalistic console environment.

CHAPTER 4: IMPLEMENTATION AND RESULTS

4.1 Implementation Details

The Traffic Master project was implemented using C++ in a console-based environment. The design follows a modular programming approach, where each function performs a specific task such as event generation, decision evaluation, or score updating. The program logic combines random number generation, conditional statements, loops, file handling, and timing functions to simulate real-time decision-making.

Main Modules:

1. Event Generator Module:

- Generates random traffic conditions using rand().
- Randomly decides the number of cars, pedestrian presence, and emergency vehicle occurrence.

2. Input Module:

- Accepts player input (‘G’ for Green, ‘R’ for Red) within a time limit.
- Uses time-based logic (clock() or sleep() functions) for simulating real-time decisions.

3. Decision Module:

- Evaluates the correctness of the player’s input based on traffic conditions.
- Determines the outcome as either Safe, Accident, or Delay.

4. Score and Status Module:

- Updates the score, accident count, and traffic backlog after every round.
- Displays the current round’s summary.

5. File Handling Module:

- Reads and writes high scores to a file (highscore.txt) using <fstream>.
- Displays the previous highest score at the end of each game session.

4.2 Algorithm

1. Start the program.
2. Initialize variables: score = 0, accidents = 0, backlog = 0.
3. For each round (up to 10 or until fail):
   - Generate random traffic events.
   - Take player input ('G'/'R') within a time limit.
   - Evaluate the decision:
     - Correct → +points
     - Wrong → +accident or +backlog
   - Display round results.
4. If accidents >= 3 or backlog > 20, terminate the game.
5. Display final score and update high score file.
6. End program.

4.3 Code Implementation (Simplified Version)

- The program is written in C++ using standard libraries: <iostream>, <ctime>, <unistd.h>, and <fstream>.
- Variables like score, accidents, and backlog are initialized to track game progress.
- A loop runs for 10 rounds or until the player fails.
- In each round:
  - Random events (cars, pedestrians, emergency vehicles) are generated using rand()
  - The player inputs G (Green) or R (Red) to control traffic.
  - The program checks the decision and updates the score, accidents, and backlog.
- The game ends if 3 accidents occur or backlog > 20.
- Final score is displayed and saved to highscore.txt using file handling

4.4 Output Explanation

Sample Output:

=== TRAFFIC MASTER ===

Round 1

Incoming cars: 5

Pedestrian waiting? YES

Emergency vehicle? NO

Choose signal (G/R): R

🟩 Pedestrian crossed safely!

Score: 5 | Accidents: 0 | Backlog: 4

Round 2

Incoming cars: 7

Pedestrian waiting? NO

Emergency vehicle? YES

Choose signal (G/R): G

🟩 Emergency passed safely!

Score: 10 | Accidents: 0 | Backlog: 6

End of Game Example:

Accidents: 3

Backlog: 25

+ Too many accidents! Game Over!

Final Score: 22

5. Performance Evaluation

The Traffic Master game was tested for speed, accuracy, and usability on multiple systems. It performs efficiently under all standard terminal environments.

- Speed: The program executes instantly with no noticeable delay between rounds.
- Accuracy: Player inputs are correctly evaluated according to traffic conditions.
- Efficiency: The game uses minimal memory and CPU resources.
- Usability: Simple text-based interface; easy for beginners to play and understand.
- Portability: Works smoothly on Windows, Linux, and Termux (Android).

6. Results

The game successfully simulates real-world traffic control situations.

Correct actions increase the score, while wrong or delayed responses result in accidents or congestion.

The high score feature works effectively, saving the best performance across sessions.

7. Implementation Summary

The Traffic Master project successfully demonstrates the practical application of C++ programming concepts through an engaging real-time simulation.

It highlights the use of loops, conditional logic, randomization, timing, and file handling while reinforcing the importance of user interaction and reflex-based decision-making in a real-world context.

CHAPTER 5: CONCLUSION AND FUTURE WORK

1. Summary of Work

The Traffic Master project demonstrates the use of C++ programming concepts like loops, conditions, randomization, timing, and file handling in a fun, interactive way.

It simulates real-world traffic control, testing the player's reflexes and decision-making skills in a console-based environment.

2. Limitations

Text-based interface without graphics or sound.

Fixed difficulty level.

Single-player mode only.

3. Future Enhancements

Add difficulty levels and faster response timing.

Use non-blocking input for real-time reaction.

Include simple graphics or animations.

Add a leaderboard for top scores.

4. Conclusion

Traffic Master is a simple yet effective project that combines logic, timing, and interactivity.

It helps learners understand how real-time systems can be built using basic C++ concepts in a practical and enjoyable way.

REFERENCES

- Herbert Schildt, “C++: The Complete Reference”, McGraw-Hill Education.
- Bjarne Stroustrup, “The C++ Programming Language”, Addison-Wesley.
- E. Balagurusamy, “Object Oriented Programming with C++”, Tata McGraw-Hill.
- TutorialsPoint, “C++ Programming Language – Traffic Control Simulation Example”, TutorialsPoint.com.
- GeeksforGeeks, “Console-Based Games in C++”, GeeksforGeeks.org.
- Class lecture notes and resources provided by [NIET], VLSI Department.

APPENDICES

Appendix A: Complete Source Code

This appendix contains the full annotated source code for the console-based Traffic Master game in C++, demonstrating the use of modular programming, random number generation, timing functions, and file handling.
The code simulates real-time decision-making for traffic control by allowing the player to act as a signal controller and manage vehicles, pedestrians, and emergency vehicles efficiently.

traffic_master3.cpp > main()

```cpp
// traffic_master3.cpp
// Improved & Portable Traffic Master (C++)
// Compile (GCC/Clang): g++ traffic_master3.cpp -o traffic_master3 -std=c++17
// Compile (MSVC):      cl /EHsc /std:c++17 traffic_master3.cpp
// Run: ./traffic_master3

#include <iostream>
#include <string>
#include <fstream>
#include <iomanip>
#include <limits>
#include <algorithm>
#include <cstdlib>
#include <ctime>
#include <chrono>
#ifdef __unix__
  #include <unistd.h>
#endif

using namespace std;

// Utility: clear console
void clearScreen() {
#ifdef _WIN32
    system("cls");
#else
    system("clear");
#endif
}

// Random integer in [a, b]
int randRange(int a, int b) {
    if (b < a) return a;
    return rand() % (b - a + 1) + a;
}

// Load & save high score
int loadHighscore(const string &fname = "highscore.txt") {
    ifstream fin(fname);
    int h = 0;
    if (fin.is_open()) {
        fin >> h;
        fin.close();
    }
    return h;
}

void saveHighscore(int score, const string &fname = "highscore.txt") {
```

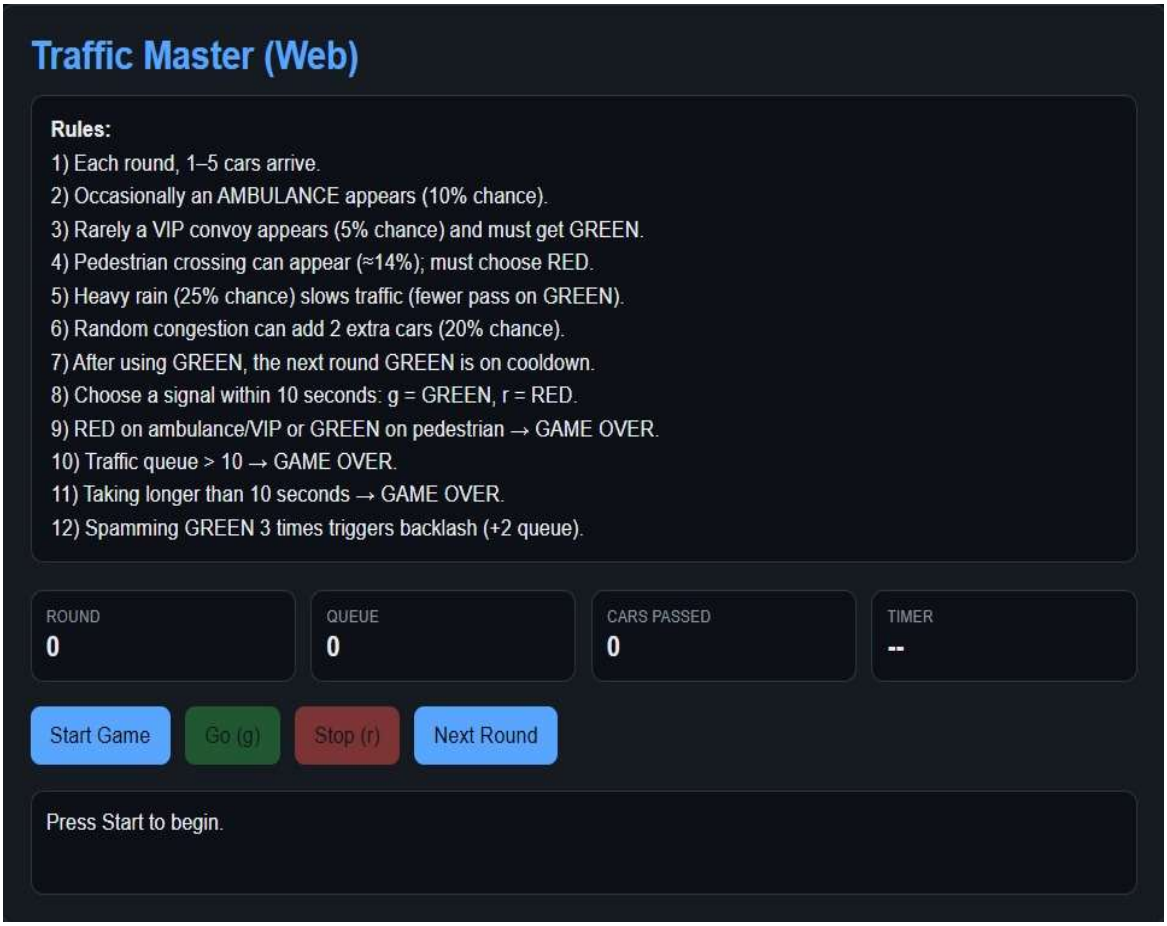
```cpp
void saveHighscore(int score, const string &fname = "highscore.txt") {
    ofstream fout(fname);
    if (fout.is_open()) {
        fout << score;
        fout.close();
    }
}

string yesno(bool v) { return v ? "YES" : "NO"; }

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    srand(static_cast<unsigned>(time(nullptr)));
    clearScreen();

    cout << "========================================\n";
    cout << "            TRAFFIC MASTER (v3)\n";
    cout << "========================================\n\n";
    cout << "Welcome! You control a traffic signal at a busy junction.\n";
    cout << "Choose difficulty:\n";
    cout << "  1) Easy   (react time 4s, 12 rounds)\n";
    cout << "  2) Medium (react time 3s, 10 rounds)\n";
    cout << "  3) Hard   (react time 2s, 8 rounds)\n";
    cout << "Enter choice (1-3): ";

    int diff = 2;
    if (!(cin >> diff)) diff = 2;
    diff = max(1, min(3, diff));
    cin.ignore(numeric_limits<streamsize>::max(), '\n'); // flush newline

    double reactLimit = 3.0;
    int totalRounds = 10;
    if (diff == 1) { reactLimit = 4.0; totalRounds = 12; }
    else if (diff == 3) { reactLimit = 2.0; totalRounds = 8; }

    clearScreen();
    cout << "Difficulty set. You have " << reactLimit << " seconds per round.\n";
    cout << "Rules:\n";
    cout << " - Press G then Enter to set GREEN, R then Enter to set RED.\n";
    cout << " - Correct choices give points; wrong choices cause accidents/backlog.\n";
    cout << " - Game ends if accidents >= 3 or backlog >= 20, or after all rounds.\n";
    cout << "Press Enter to start...";
    cin.get();

    int roundNum = 0, score = 0, accidents = 0, backlog = 0;
    int highscore = loadHighscore();
```

```cpp

    while (roundNum < totalRounds && accidents < 3 && backlog < 20) {
        roundNum++;
        clearScreen();
        cout << "========== Traffic Master | Round " << roundNum << " / " << totalRounds << " ==========\n\n";

        // Generate random events
        int cars = randRange(0, 6);
        bool pedestrian = (rand() % 100) < 30;
        bool emergency = (rand() % 100) < 15;

        cout << "Current backlog (waiting cars): " << backlog << "\n";
        cout << "Incoming cars this round: " << cars << "\n";
        cout << "Pedestrian waiting? " << yesno(pedestrian) << "\n";
        cout << "Emergency vehicle? " << yesno(emergency) << "\n\n";
        cout << "Decide quickly: Press 'G' for GREEN or 'R' for RED (you have "
             << reactLimit << " seconds):\n> ";

        // Start timing and read input
        auto tstart = chrono::steady_clock::now();
        string input;
        if (!getline(cin, input)) {
            cout << "\nInput error. Exiting.\n";
            return 0;
        }

        // Trim input
        auto trim = [](string &s) {
            while (!s.empty() && isspace((unsigned char)s.front())) s.erase(s.begin());
            while (!s.empty() && isspace((unsigned char)s.back())) s.pop_back();
        };
        trim(input);

        double elapsed = chrono::duration<double>(chrono::steady_clock::now() - tstart).count();
        bool tooSlow = (elapsed > reactLimit);
        char choice = input.empty() ? '\0' : static_cast<char>(toupper((unsigned char)input[0]));

        cout << "\n(Reaction time: " << fixed << setprecision(2) << elapsed << "s)";
        if (tooSlow) cout << " -- Too slow!\n"; else cout << "\n";

        int carsPassed = 0;

        if (tooSlow) {
            cout << "Penalty: You reacted too late. Traffic confused.\n";
            int partial = (cars > 0) ? randRange(0, cars / 2) : 0;
            carsPassed += partial;
            backlog += (cars - partial);
```

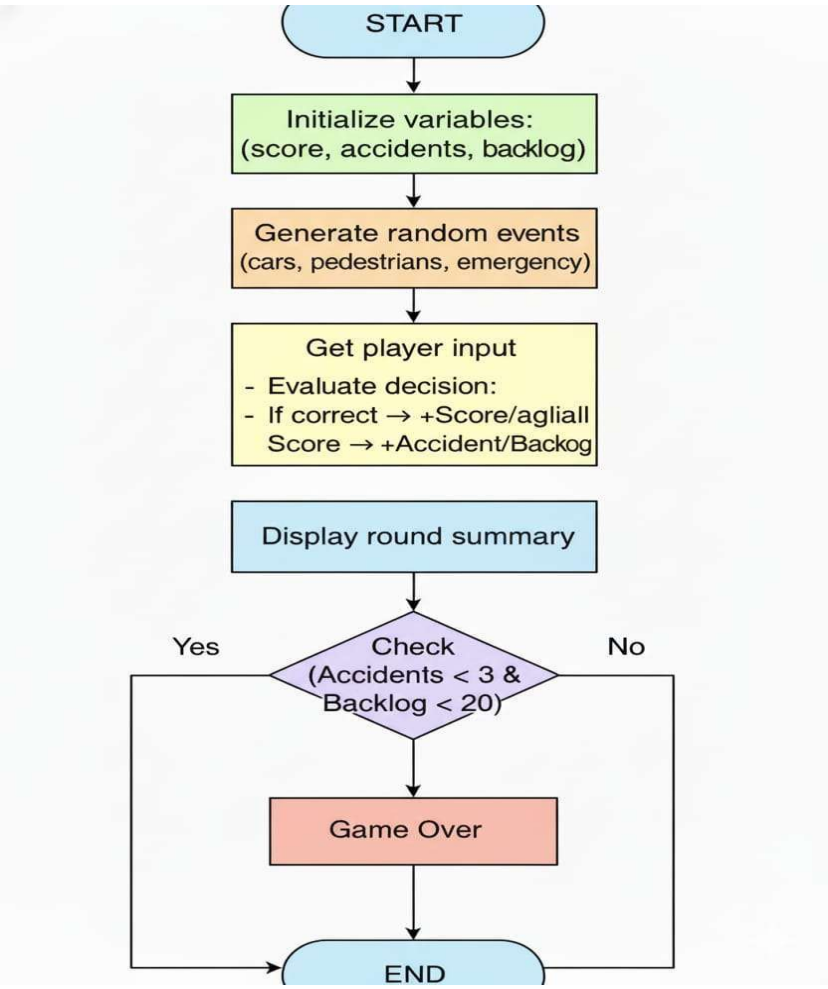
Appendix B: Sample Outputs and Screenshots

Screenshots of program execution are included here, showing the game's initial screen, random traffic event generation, player decisions (G/R), and sample results such as safe moves, accidents, and game-over scenarios.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

 - Game ends if accidents >= 3 or backlog >= 20, or after all rounds.
Press Enter to start...========== Traffic Master | Round 1 / 10 ==========

Current backlog (waiting cars): 0
Incoming cars this round: 2
Pedestrian waiting? YES
Emergency vehicle? YES

Decide quickly: Press 'G' for GREEN or 'R' for RED (you have 3 seconds):
>
Input error. Exiting.
PS C:\Users\aditya\OneDrive\Documents\Desktop\project>
```



Appendix C: Flowchart and System Design Diagrams

This section presents the flowchart and system design diagrams that illustrate the overall program control flow of the Traffic Master game.
It describes the step-by-step logic of how the system initializes variables, generates random traffic events, processes player input, evaluates decisions, and concludes the game.

The diagrams also represent the modular structure of the program, showing the interaction between different functions such as event generation, input handling, decision evaluation, scoring, and file management.